

Lab 01

Object Oriented Programming

BS DS/CY Fall 2025 Morning/Afternoon

Instructions:

- Indent your code properly.
- Use meaningful variable names and follow standard naming conventions.
- Use clear and meaningful prompts/labels for all program input and output.
- Do not discuss your problems with fellow students. If you face any difficulty in understanding, consult your TAs.
-

Task: 01

Write a program that contains following functions:

- 1- **bool addArrays(int arr1[], int s1, int arr2[], int s2)** it takes two arrays and their sizes and if the sizes of both arrays are same then adds arr2 into arr1 and returns true otherwise it does not perform addition and returns false.
- 2- **bool addArrays(int arr1[[..]], int r1, int c1, int arr2[[..]], int r2, int c2)** it takes two matrices and their dimensions(i.e. number of rows and columns) and if the dimensions of both 2D arrays are same then adds arr2 into arr1 and returns true otherwise it does not perform addition and returns false.
- 3- **int main()** it contains required variables and calls the above two functions and displays proper messages.

Task: 02

Write a function that will take an array of **Characters** as parameters and returns true if it is a Palindrome and return false if it is not Palindrome.

Sample Input: Enter a Character Array : abba Sample output: It is a Palindrome!	Sample Input: Enter a Character Array : 12345678 Sample output: It is NOT a Palindrome!
---	---

Task: 03

Write a program to print all the pair of elements of the given integer array, where smaller element divides the largest element. Do not change the original order of the elements.

Input: 12 8 9 5 2 20 6 3

Output: 12 2 - 12 6 - 12 3 - 8 2 - 9 3 - 5 20 - 2 30 - 6 3

Task: 04

Write a program to rearrange the array so that all negative numbers appear before positive numbers, while maintaining the original order as much as possible.

Sample Input: 8 12 -7 5 -1 -2 8 -3 6

Sample Output: -7 -1 -2 -3 12 5 8 6

Task: 05

2D Array Operations

Write a program that creates a two-dimensional array initialized with test data. Use any data type you wish. The program should have the following functions:

- **getTotal**. This function should accept a two-dimensional array as its argument and return the total of all the values in the array.

- **getAverage**. This function should accept a two-dimensional array as its argument and return the average of all the values in the array.

- **getRowTotal**. This function should accept a two-dimensional array as its first argument and an integer as its second argument. The second argument should be the subscript of a row in the array. The function should return the total values in the specified row.

- **getColumnTotal**. This function should accept a two-dimensional array as its first argument and an integer as its second argument. The second argument should be the subscript of a column in the array. The function should return the total of the values in the specified column.

- **getHighestInRow**. This function should accept a two-dimensional array as its first argument and an integer as its second argument. The second argument should be the subscript of a row in the array. The function should return the highest value in the specified row of the array.

- **getLowestInRow**. This function should accept a two-dimensional array as its first argument and an integer as its second argument. The second argument should be the subscript of a row in the array. The function should return the lowest value in the specified row of the array.

Demonstrate each of the functions in this program.

Bonus Task:

Write a program to rotate a given square matrix by 90 degrees in the clockwise direction.

Input:

A square matrix of size n x n

Output:

The resulting matrix after rotating the original matrix 90 degrees clockwise.

Example:

Input Matrix:

```
1 2 3
4 5 6
7 8 9
```

Output Matrix:

```
7 4 1
8 5 2
9 6 3
```